

ADR-001 — División del backend en dos servicios desplegados independientes

ID	Título	Estado	Fecha
ADR-001	División del backend en dos servicios desplegados independientes	Aceptada	27/05/2026

1. Contexto

Polizing tiene dos audiencias claramente distintas que entran al sistema por canales distintos:

- **Productores y administrativos** usan un panel web para gestionar clientes, pólizas, siniestros y pagos durante su jornada laboral. Es un CRUD pesado con formularios y reportes.
- **Clientes finales** interactúan exclusivamente por WhatsApp para pedir su tarjeta de circulación, cargar comprobantes de pago o reportar siniestros. No tienen interfaz web propia.

Los motivadores que obligan a tomar una decisión arquitectónica:

- **Motivador 1 — Mantenibilidad (RNF Entrega 1):** los dos canales evolucionan a ritmos diferentes. El equipo necesita poder cambiar el flujo del bot (agregar una opción de menú, ajustar un texto) sin tocar ni desplegar el panel web, y viceversa. La normativa aseguradora cambia con frecuencia y forzar un único pipeline ralentiza ambos lados.
- **Motivador 2 — Disponibilidad 24/7 (RNF Entrega 1):** el chatbot debe seguir respondiendo aunque el panel web esté caído en una ventana de mantenimiento o liberando una versión. Un siniestro reportado a las 3 AM no puede depender de que un deploy del panel haya terminado bien.
- **Motivador 3 — Restricciones de runtime:** la integración con WhatsApp Cloud API requiere mantener un webhook público estable, sesiones HTTP largas para envío de medios y un proceso con estado de conversación en memoria/BD local. El stack del panel (Next.js sobre serverless) no encaja con ese perfil; el stack del bot (FastAPI sobre proceso persistente) no encaja con el de un panel React server-rendered.

2. Alternativas consideradas

Alternativa	Ventaja principal en este contexto	Desventaja / Motivo de descarte
Monolito único en Next.js (chatbot embebido como API routes)	Un solo deploy, un solo repo, un solo lenguaje (TypeScript). Modelo de datos compartido sin duplicación.	El webhook de WhatsApp y el manejador de sesiones conversacionales no encajan bien en un runtime serverless: pierden contexto entre invocaciones y obligan a workarounds caros. Acopla los ciclos de release de panel y bot, contradiciendo el motivador 1.
Microservicios completos (un servicio por dominio: clientes, pólizas, siniestros, pagos, bot, notificaciones)	Máxima independencia de despliegue por dominio; cada servicio puede escalar y evolucionar solo.	Sobreingeniería para un equipo de 6 personas en un PoC académico. Multiplica overhead de infraestructura (descubrimiento, observabilidad, contratos entre servicios) sin un volumen que lo justifique.

Dos servicios separados: panel web + chatbot (elegida)	Cada servicio usa el stack que mejor le sirve, despliegues independientes, fallo aislado por canal.	— (esta fue la elegida)
--	---	-------------------------

3. Decisión tomada

Se decide: dividir el backend en **dos servicios desplegables independientes** — `my-app` (panel web Next.js + API REST de negocio) y `chatbot` (servicio FastAPI que orquesta WhatsApp)— que se comunican entre sí únicamente por HTTP. Cada uno vive en su propio directorio del monorepo, con su propio stack, sus propias dependencias y su propio pipeline.

Fundamentación:

- Resuelve el motivador 1 (mantenibilidad y ritmo de evolución):** un cambio de copy en el bot o el agregado de una opción del menú se libera sin tocar `my-app`. Un refactor del modelo de siniestros en el panel no requiere redespargar el bot. Los equipos de cada lado pueden trabajar en paralelo.
- Resuelve el motivador 2 (disponibilidad 24/7):** si una versión del panel sale rota, el chatbot sigue tomando mensajes y encolando comprobantes/siniestros que se materializan en `my-app` cuando vuelve. La inversa también vale: si el bot cae, el panel sigue operativo para el productor.
- Resuelve el motivador 3 (runtimes incompatibles):** Next.js sobre Vercel resuelve bien el panel (server components, preview deploys, edge caché), mientras que FastAPI sobre un proceso persistente resuelve bien el bot (webhook estable, estado conversacional en BD local, librerías Python para integraciones futuras). Forzar un único runtime hubiera obligado a comprometer uno de los dos.

4. Consecuencias

Consecuencias positivas	Trade-offs / costos
Cada servicio elige su stack óptimo (Next.js + Prisma para el panel; FastAPI + SQLAlchemy para el bot).	Aparece acoplamiento por contrato HTTP : cualquier cambio en endpoints expuestos por <code>my-app</code> que el bot consume rompe al bot si no se versiona la API.
Despliegues independientes: el ritmo del bot no está atado al ritmo del panel.	El modelo de dominio se duplica parcialmente (el bot necesita su propia representación de cliente, póliza, siniestro en SQLAlchemy, espejo de la del panel).
Aislamiento de fallos por canal: un bug en el panel no tira la atención al cliente final.	Dos pipelines de CI/CD, dos paneles de observabilidad, dos lugares donde rotar secretos (ver ADR-004).
Encaja con la estructura actual del repo (<code>/my-app</code> , <code>/chatbot</code>) sin forzar reorganización.	Latencia de red en cada interacción bot → panel (ver ADR-003 para cómo se mitiga).

Decisiones relacionadas:

- ADR-002 (persistencia en PostgreSQL): aplica al panel; el bot mantiene su propio store local consistente con esta separación.

- ADR-003 (comunicación REST síncrono con API Key): es la consecuencia directa de tener dos servicios que necesitan hablarse.
- ADR-004 (despliegue heterogéneo): los runtimes distintos obligan a infraestructura distinta para cada servicio.